Proximie Senior ML Engineer Challenge

Proximie AI Team

July 6, 2026

1 Challenge Overview

At Proximie, we are transforming surgical collaboration globally. A key pillar of our long-term vision is understanding the complex workflow of the Operating Room (OR) in real time. This challenge combines strategic system design with hands-on implementation to address live **action segmentation** under strict real-world data and cloud constraints.

This exercise evaluates both your high-level engineering judgment and your coding hygiene. Rather than asking you to train massive production networks, we look for structural reproducibility, clean evaluation setups, and deep reasoning around architectural trade-offs.

- **Estimated Effort:** ∼7 hours total.
- **Deliverables Required:**
  1. A structured **Git repository** containing your prototype pipeline code.
  2. A concise **Technical Architecture Report** (3–5 pages, integrated or submitted as PDF).

- **Dataset Recommendation for Inspiration**: While you are expected to use mock features to ensure the script runs standalone, we recommend reviewing open-source benchmarks like MM-OR (`github.com/egeozsoy/MM-OR`) for structural inspiration.
- **Candidates are encouraged to use AI tools**, but they must maintain complete mastery over 100% of the resulting code and content.

2 The Scenario & Technical Constraints

2.1 The Infrastructure & Feed

- **Sensors:** Every operating theater is pre-equipped with **1–3 optical sensors** permanently mounted in the corners of the room, providing multi-angle coverage.
- **The Feed:** To balance network bandwidth costs across thousands of hospitals, the cameras capture and stream.
- **The Cloud: All model inference, processing, and multi-stream assembly run entirely in AWS**. The streams are pushed dynamically from the hospital gateway directly to AWS endpoints in real time.

2.2 The Data Retention Bottleneck

Due to strict hospital privacy agreements and data governance policies, **Proximie operates under a strict N-day data retention threshold** (N could be between 30 and 360). New live streams cannot be persistently cached or stored as raw video feeds for manual retrospective labeling or failure analysis.

2.3 The Performance Bottleneck

Current action segmentation baseline experiences severe degradation on two critical workflow classes:

- **"Patient Present":** A long-duration, highly static phase heavily impacted by background noise (e.g., staff movement, equipment adjustments). It suffers from frequent false positives and segment fragmentation before the surgery begins.
- **"Operation":** A high-intensity, occlusion-heavy phase where the core surgical action takes place. The model struggles to detect precise temporal boundaries (start/end) and suffers from frequent camera occlusions.

3 Deliverable 1: The Code (Prototype Pipeline)

You are asked to deliver a clean, structured Python repository containing a mock/lightweight pipeline. The objective is to demonstrate how you structure a temporal modeling code-base. You do not need to optimize for accuracy; focus on pipeline reproducibility and design.

Your repository must include:

1. **Ingestion & Feature Stubs:** A module simulating the ingestion of pre-computed visual features from multiple sensor streams. You may use mock data generators to substitute for real video features.
2. **Temporal Classification Layer:** A lightweight temporal sequence model that accepts these features over time and predicts workflow phases.
3. **Segmentation Timeline Generator:** Post-processing logic that smooths out frame-level outputs and produces a clean, contiguous timeline of predicted activity segments (collapsing frame-level noise).
4. **Dual Metric Stack:** An evaluation suite calculating both model and product quality metrics
5. **Mock Error Analysis Script:** A routine simulating an evaluation run on a noisy validation set, printing a short breakdown of where predictions break down on the **"Patient Present"** and **"Operation"** classes.

4 Deliverable 2: Technical Architecture Report

Complementing your code, write a structured design report answering the following deep-dive system questions:

4.1 Data Strategy & Annotation Lifecycle

1. **In-Flight Ingestion:** How do you capture, flag, or distill "hard examples" for underperforming classes inside the live streaming AWS pipeline without violating the data retention policy? Discuss streaming anonymized feature extractions or vector embeddings.
2. **Active Learning on Terabytes of Historical Data:** We possess Terabytes of historical unlabelled video, but clinical labeling is highly expensive. How would you design an offline data curation pipeline to selectively sample the most informative frames or clips to send to human annotators?
3. **Annotation Optimization:** Suggest a method (e.g., self-supervised pre-training, semi-supervised loops) to minimize manual annotation costs for temporal streams.

4.2 Modeling & Multi-Sensor Fusion

1. **Multi-View Spatial Fusion:** Explain how you will geometrically or temporally fuse the synchronous streams from the 2–3 corner sensors to handle severe body/equipment occlusions

during the **"Operation"** phase.
2. **Temporal Scale Balancing:** Propose a network architecture capable of resolving both long-form macro contexts (**"Patient Present"**) and rapid, fine-grained state transitions (**"Operation"**) under strict sampling constraint.

4.3 AWS Cloud Architecture & Cost Optimization

1. **The Scale Architecture:** Provide a cloud architecture diagram mapping out real-time ingestion, frame synchronization/jitter management across multi-camera streams, and live model inference. Specify concrete AWS services.
2. **Cost Optimization Strategy:** Running large deep learning architectures 24/7 across hundreds of live operating rooms is cost-prohibitive. Outline strategies (e.g., frame-skipping when empty, dynamic serverless scaling, or batched inference endpoints) to prevent ballooning AWS bills.

5 Evaluation Criteria

Your submission will be reviewed holistically by our AI leadership team on:

- **Code Hygiene:** Proper separation of concerns, clean interfaces, modularity, and reproducible configuration.
- **Architectural Depth:** Realistic consideration of FPS constraints, multi-camera geometry, and cloud costs.
- **Product Awareness:** Your approach to metrics that bridge the gap between machine learning performance and ultimate clinical utility.